Lab Manual

Programming for AI
AI-212

By

M. Faisal Kamran 2023-BSAI-025

Submitted to

Mr. Samraiz

Bachelor of Science in Artificial Intelligence

DEPARTMENT OF COMPUTER SCIENCES

Table of Contents

Python Basics

Objectives

The primary objective of this programming lab is to strengthen the understanding and practical application of **Python programming fundamentals** through real-world problem scenarios. This lab is designed to evaluate the student's ability to analyze problems, apply logical thinking, and implement solutions using **basic programming constructs** without relying on Object-Oriented Programming concepts.

Specifically, this lab aims to:

1. **Develop Logical Thinking Skills**
   Enable students to break down real-life systems such as banking, healthcare, e-commerce, and inventory management into smaller logical steps and implement them using conditional statements and loops.
2. **Reinforce Core Python Concepts**
   Provide hands-on practice with Python basics including:
   - Variables and data types
   - Arithmetic and logical operators
   - Conditional statements (if, elif, else)
   - Loops (for, while)
   - Functions with parameters and return values
   - Data structures such as **lists, tuples, and dictionaries**
3. **Understand Decision-Based Programming**
   Train students to implement multi-condition decision-making systems such as grading systems, medical triage, billing slabs, discount engines, and eligibility checks using nested conditions.
4. **Apply Mathematical and Business Logic**
   Help students implement calculations involving percentages, tax, discounts, penalties, weightage, service charges, and statistical analysis using operators and expressions.
5. **Simulate Real-World Systems**
   Allow students to model and simulate common real-world applications such as:
   - ATM and banking transactions
   - Grocery and vending machine systems

- Hotel and flight booking systems
- Examination grading and scholarship eligibility
- Utility billing and recharge systems

6. **Promote Input Validation and Error Handling Awareness**
   Introduce the importance of validating user input and handling invalid cases such as insufficient balance, incorrect PIN attempts, out-of-stock items, and failed eligibility conditions.
7. **Encourage Modular Programming**
   Familiarize students with writing reusable and organized code using functions for specific tasks like bill calculation, score prediction, and fare computation.
8. **Prepare for Advanced Programming Concepts**
   Build a strong foundation in procedural programming that prepares students for future learning in **Object-Oriented Programming, Data Structures, and Software Development**.

Expected Learning Outcomes

After completing this lab, students will be able to:

- Write correct and readable Python programs using basic constructs
- Implement real-world logic using conditions and loops
- Use Python data structures effectively
- Create simple function-based programs
- Analyze problems and convert them into executable algorithms

Code Examples

Q1. Banking Transaction Validator with PIN
Simulate a bank ATM.
User enters a 4-digit PIN (check if valid).
Display account balance and ask for withdrawal amount.
If valid → deduct + update balance.
If invalid attempts ≥ 3 → block card.
Use operators to compute service charges (2% per withdrawal).

Code

```
balance = 50000
correct_pin = "1234"
attempts = 0

while attempts < 3:
    pin = input("Enter 4-digit PIN: ")
    if pin == correct_pin:
        print("Balance:", balance)
        amount = int(input("Enter withdrawal amount: "))
        service_charge = amount * 0.02
        total = amount + service_charge

        if total <= balance:
            balance -= total
            print("Withdrawal successful")
            print("Service Charge:", service_charge)
            print("Remaining Balance:", balance)
        else:
            print("Insufficient balance")
        break
    else:
        attempts += 1
        print("Invalid PIN")

if attempts == 3:
    print("Card Blocked")
```

✓ 7.9s

```
Balance: 50000
Withdrawal successful
Service Charge: 100.0
Remaining Balance: 44900.0
```

Q2. Grocery Store Inventory System

Maintain item names, stock, and prices in a dictionary of dictionaries.
User selects items with quantity.
If stock available, deduct from inventory.
Apply discount slabs (10%, 15%, 20%).
Print final bill with tax (5%).

Code

```
inventory = {
    "apple": {"stock": 50, "price": 100},
    "banana": {"stock": 100, "price": 50},
    "milk": {"stock": 30, "price": 150}
}

total = 0
item = input("Enter item: ")
qty = int(input("Enter quantity: "))

if item in inventory and inventory[item]["stock"] >= qty:
    inventory[item]["stock"] -= qty
    total = inventory[item]["price"] * qty
else:
    print("Item not available")

if total >= 500:
    discount = 0.20
elif total >= 300:
    discount = 0.15
else:
    discount = 0.10

total -= total * discount
tax = total * 0.05
print("Final Bill:", total + tax)
```

✓ 6.8s

```
Final Bill: 588.0
```

Q3. Student Grading with Subject Weightage

Input marks for 5 subjects where each subject has different weightage (stored in a tuple).
Compute weighted average.
Assign grade (A/B/C/D/Fail).
Also, display whether the student is eligible for scholarship (≥80% weighted).

Code

```
weights = (0.2, 0.2, 0.2, 0.2, 0.2)
marks = []
weighted_sum = 0

for i in range(5):
    m = int(input(f"Enter marks for subject {i+1}: "))
    marks.append(m)
    weighted_sum += m * weights[i]

percentage = weighted_sum

if percentage >= 80:
    grade = "A"
elif percentage >= 65:
    grade = "B"
elif percentage >= 50:
    grade = "C"
elif percentage >= 40:
    grade = "D"
else:
    grade = "Fail"

print("Grade:", grade)
print("Scholarship:", percentage >= 80)
```

✓ 5.2s

```
Grade: Fail
Scholarship: False
```

Q4. BMI + Health Recommendation

Extend BMI program:
Take weight, height, and age.
Compute BMI.
Based on BMI and age, give personalized health suggestions (e.g., diet plan for obese, exercise plan for underweight).

Code

```
weight = float(input("Weight (kg): "))
height = float(input("Height (m): "))
age = int(input("Age: "))

bmi = weight / (height ** 2)
print("BMI:", bmi)

if bmi < 18.5:
    print("Underweight: Eat healthy and exercise lightly")
elif bmi < 25:
    print("Normal: Maintain diet")
elif bmi < 30:
    print("Overweight: Exercise regularly")
else:
    print("Obese: Strict diet and doctor consultation")
```

✓ 5.4s

```
BMI: 0.08641975308641975
Underweight: Eat healthy and exercise lightly
```

Q5. Telecom Prepaid Recharge System

Maintain call rate/min, SMS rate, and data/MB in a dictionary.
User selects usage (calls, SMS, data).
Deduct accordingly from balance.
If balance < 20% of initial → print recharge reminder.
Apply bonus balance if recharge > 1000.

Code

```
rates = {"call": 2, "sms": 1, "data": 0.5}
balance = 1000
initial = balance

calls = int(input("Call minutes: "))
sms = int(input("SMS count: "))
data = int(input("Data MB: "))

balance -= (calls * rates["call"] + sms * rates["sms"] + data * rates["data"])

if balance < initial * 0.2:
    print("Recharge Reminder")

recharge = int(input("Recharge amount: "))
if recharge > 1000:
    recharge += 100

balance += recharge
print("Final Balance:", balance)
```

✓ 8.6s

```
Final Balance: 950.0
```

Q6. E-Commerce Discount Engine with Membership Levels

User inputs bill amount + membership type (Regular, Gold, Platinum).
Apply different discount rules (Gold +10%, Platinum +15%).
If bill > 10000, apply additional 5% flat.
Show itemized receipt with tax breakdown.

Code

```
amount = float(input("Bill amount: "))
member = input("Membership (Regular/Gold/Platinum): ")

discount = 0
if member == "Gold":
    discount = 0.10
elif member == "Platinum":
    discount = 0.15

amount -= amount * discount

if amount > 10000:
    amount -= amount * 0.05

tax = amount * 0.05
print("Final Bill:", amount + tax)
```

✓ 12.0s

```
Final Bill: 189.0
```

Q7. Hospital Emergency Unit with Multi-Condition Triage

Take patient details: temperature, pulse, oxygen, blood pressure.
Classify into categories: Stable, Observation, Urgent, Critical.
If critical → suggest "Admit in ICU."
If urgent → suggest "Immediate doctor attention."
Use nested conditions.

Code

```
temp = float(input("Temperature: "))
pulse = int(input("Pulse: "))
oxygen = int(input("Oxygen: "))
bp = int(input("BP: "))

if oxygen < 85 or bp < 90:
    print("Critical → Admit in ICU")
elif oxygen < 92 or temp > 102:
    print("Urgent → Immediate Doctor")
elif pulse > 100:
    print("Observation")
else:
    print("Stable")
```

✓ 8.2s

```
Critical → Admit in ICU
```

Q8. Online Examination Grader with Negative Marking

Take total marks, obtained marks, and number of wrong answers.
Deduct 0.25 per wrong answer.
Compute percentage.
Assign division + eligibility for scholarship.
If Fail → suggest "Repeat Exam."

Code

```
total = int(input("Total Marks: "))
obtained = float(input("Obtained Marks: "))
wrong = int(input("Wrong Answers: "))

obtained -= wrong * 0.25
percentage = (obtained / total) * 100

print("Percentage:", percentage)

if percentage >= 60:
    print("Division: First")
    print("Scholarship Eligible")
elif percentage >= 45:
    print("Division: Second")
else:
    print("Fail")
    print("Repeat Exam")
```

```
Percentage: 100.0
Division: First
Scholarship Eligible
```

Q9. Movie Ticket Booking with Age & Timing Policy

User inputs: age, showtime, ticket type (Normal/3D/IMAX).
Apply child restrictions (No late-night for <12).
Apply senior citizen discount (30%).
Apply peak hour charges (10%).
Print final ticket price.

Code

```
age = int(input("Age: "))
time = int(input("Showtime (24hr): "))
ticket = input("Ticket Type (Normal/3D/IMAX): ")

price = 300 if ticket == "Normal" else 500 if ticket == "3D" else 700

if age < 12 and time >= 22:
    print("Late-night shows not allowed")
else:
    if age >= 60:
        price *= 0.7
    if 18 <= time <= 21:
        price *= 1.1
    print("Final Ticket Price:", price)
```

```
Final Ticket Price: 300
```

Q10. Weather-based Outfit + Travel Suggestion

Input temperature, weather (Rainy/Sunny/Cold), event type (Casual/Business/Party).
Suggest outfit + accessories.
If Rainy → umbrella; If Cold → gloves.
If event is Business + temperature > 35 → recommend "light formal suit."

Code

```
temp = int(input("Temperature: "))
weather = input("Weather: ")
event = input("Event Type: ")

if weather == "Rainy":
    print("Carry Umbrella")
if weather == "Cold":
    print("Wear Gloves")

if event == "Business" and temp > 35:
    print("Light formal suit")
else:
    print("Casual outfit recommended")
```

✓ 22.5s

```
Wear Gloves
Light formal suit
```

Q11. ATM Withdrawal with Note Counting + Receipt

Take withdrawal amount.
Compute minimum number of notes (1000, 500, 100, 50, 10, 1).
Also compute service charge = 0.5%.
Print receipt with remaining balance after deduction.

Code

```
amount = int(input("Withdrawal Amount: "))
balance = 50000

notes = [1000, 500, 100, 50, 10, 1]
print("Notes Breakdown:")


for n in notes:
    print(n, ":", amount // n)
    amount %= n

charge = balance * 0.005
balance -= charge

print("Service Charge:", charge)
print("Remaining Balance:", balance)
```

```
Notes Breakdown:
1000 : 0
500 : 0
100 : 0
50 : 1
10 : 0
1 : 6
Service Charge: 250.0
Remaining Balance: 49750.0
```

Q12. Library Fine Calculator with Membership Levels

Input days late.
Apply fines (10, 20, 50/day).
If days > 30 → "Membership Cancelled."
If user is Premium member, reduce fine by 50%.
Print detailed fine slip.

Code

```
days = int(input("Days Late: "))
member = input("Membership (Normal/Premium): ")

fine = 10 if days <= 10 else 20 if days <= 20 else 50
total = fine * days

if member == "Premium":
    total *= 0.5

if days > 30:
    print("Membership Cancelled")
else:
    print("Total Fine:", total)
```

✓ 4.3s

```
Total Fine: 50
```

Q13. Sales Analysis Dashboard

Store monthly sales in a list (12 entries).
Print max, min, average.
Print months above average.
Use loop + condition to print "Growth" or "Decline" compared to last month.

Code

```
sales = [120, 130, 125, 140, 150, 145, 160, 170, 165, 180, 175, 190]

avg = sum(sales) / 12
print("Max:", max(sales))
print("Min:", min(sales))
print("Average:", avg)

for i in range(1, 12):
    if sales[i] > sales[i-1]:
        print("Month", i+1, ": Growth")
    else:
        print("Month", i+1, ": Decline")
```

✓ 0.0s

```
Max: 190
Min: 120
Average: 154.16666666666666
Month 2 : Growth
Month 3 : Decline
Month 4 : Growth
Month 5 : Growth
Month 6 : Decline
Month 7 : Growth
Month 8 : Growth
Month 9 : Decline
Month 10 : Growth
Month 11 : Decline
Month 12 : Growth
```

Q14. Multiplication Puzzle with Random Blanks

Generate a multiplication table (1–10).
Randomly replace 3 results with "??".
Ask user to guess values.
Track score.
Give performance remark (Excellent/Good/Try Again).

Code

```
import random

score = 0
for i in range(1, 6):
    a = random.randint(1,10)
    b = random.randint(1,10)
    ans = a * b
    print(a, "x", b, "= ??")
    user = int(input("Guess: "))
    if user == ans:
        score += 1

print("Score:", score)
if score >= 4:
    print("Excellent")
elif score >= 2:
    print("Good")
else:
    print("Try Again")
```

✓ 6.7s

```
9 x 10 = ??
7 x 4 = ??
3 x 6 = ??
5 x 1 = ??
1 x 9 = ??
Score: 1
Try Again
```

Q15. Password Strength + Re-entry Attempts

Check password rules (length, digit, upper, lower, special).
If weak → ask user to re-enter.
Allow max 3 attempts.
If still weak → "Account Locked."

Code

```
attempts = 0

while attempts < 3:
    pwd = input("Enter Password: ")
    if (len(pwd) >= 8 and any(c.isupper() for c in pwd) and
        any(c.islower() for c in pwd) and any(c.isdigit() for c in pwd)):
        print("Strong Password")
        break
    else:
        print("Weak Password")
    attempts += 1

if attempts == 3:
    print("Account Locked")
```

✓ 11.5s

```
eak Password
eak Password
eak Password
ccount Locked
```

Q16. Electricity Bill with Slabs & Penalty

Function calculate_bill(units, late_payment=False):
Apply slab rates (5/7/10 Rs).
If late_payment → add 15% penalty.
Return bill breakdown.

Code

```
def calculate_bill(units, late_payment=False):
    if units <= 100:
        bill = units * 5
    elif units <= 200:
        bill = units * 7
    else:
        bill = units * 10

    if late_payment:
        bill *= 1.15
    return bill

print("Bill:", calculate_bill(250, True))
```

✓ 0.0s

```
Bill: 2875.0
```

Q17. Cricket Match Predictor with Multiple Conditions

Function predict_score(runs, balls, wickets, overs_left):
Compute strike rate & run rate.
If run rate < 5 and wickets > 6 → “Losing.”
If run rate > 7 and wickets < 5 → “Winning.”
Else → “Balanced.”

Code

```
def predict_score(runs, balls, wickets, overs_left):
    run_rate = runs / (balls/6)

    if run_rate < 5 and wickets > 6:
        print("Losing")
    elif run_rate > 7 and wickets < 5:
        print("Winning")
    else:
        print("Balanced")

predict_score(150, 120, 4, 10)
```

✓ 0.0s

```
Winning
```

Q18. Hotel Booking with Tax & Discount

Function book_room(type, days, is_member):
Standard: 2000/day, Deluxe: 4000/day, Suite: 6000/day.
If days > 7 → 20% discount.
If member → extra 10% discount.
Add 12% tax.
Return bill slip.

Code

```
def book_room(room, days, member):
    rates = {"Standard":2000, "Deluxe":4000, "Suite":6000}
    bill = rates[room] * days

    if days > 7:
        bill *= 0.8
    if member:
        bill *= 0.9

    bill *= 1.12
    return bill

print("Total Bill:", book_room("Deluxe", 8, True))
```

✓ 0.0s

```
Total Bill: 25804.800000000003
```

Q19. Flight Fare Calculator with International Tax

Function calculate_fare(distance, class_type, international=False):
Economy: 10/unit, Business: 25/unit, First: 40/unit.
Add fuel surcharge 500.
If international → add 18% tax.
If booking in advance (>30 days) → 10% discount.

Code

```
def calculate_fare(distance, class_type, international=False):
    rates = {"Economy":10, "Business":25, "First":40}
    fare = distance * rates[class_type] + 500

    if international:
        fare *= 1.18
    return fare

print("Fare:", calculate_fare(300, "Business", True))
```

✓ 0.0s

```
Fare: 9440.0
```

Q20. Smart Vending Machine with Wallet System

Function vending_machine(item, amount_paid, wallet_balance):
Items stored in dict with stock + price.
If sufficient → dispense + return change.
Update wallet balance.
If stock = 0 → suggest alternative item.

Code

```
def vending_machine(item, paid, wallet):
    items = {"Chips":50, "Soda":40, "Juice":60}

    if item not in items:
        print("Item not found")
    elif paid < items[item]:
        print("Insufficient amount")
    else:
        wallet -= items[item]
        print("Dispensed:", item)
        print("Change:", paid - items[item])
        print("Wallet Balance:", wallet)

vending_machine("Soda", 100, 500)
```

✓ 0.0s

```
Dispensed: Soda
Change: 60
Wallet Balance: 460
```

Python OOPs Concepts

Objectives

The primary objective of this programming lab is to strengthen the understanding and practical application of Object-Oriented Programming (OOP) concepts in Python through real-world problem scenarios. This lab is designed to evaluate the student's ability to design, implement, and manage programs using classes and objects, promoting modularity, reusability, and scalability in software development.

Main Concepts of OOPs

1. Class

A class is a blueprint or template used to create objects. It defines the properties (variables) and behaviors (methods) that the objects will have.

Example idea:
A Student class can define attributes like name and roll number, and methods like displaying details.

2. Object

An object is an instance of a class. It represents a real-world entity created using the class.

Example idea:
If Student is a class, then each student you create from it is an object.

3. Encapsulation

Encapsulation means binding data (variables) and methods (functions) together inside a class and restricting direct access to some of the data.

- Achieved using private variables and getter/setter methods
- Helps protect data from unauthorized access

Example idea:
A bank account hides the balance and allows access only through deposit and withdraw methods.

4. Abstraction

Abstraction focuses on showing only essential features and hiding implementation details.

- Implemented using abstract classes and abstract methods
- Helps reduce complexity

Example idea:
An ATM shows options like withdraw and check balance but hides how the bank processes them.

5. Inheritance

Inheritance allows one class (child class) to acquire the properties and methods of another class (parent class).

- Promotes code reuse
- Creates a parent-child relationship

Example idea:
A SavingsAccount class inherits from a BankAccount class.

6. Polymorphism

Polymorphism means “many forms.” It allows the same method name to behave differently based on the object.

- Achieved through method overriding
- Increases flexibility

Example idea:
A calculate_salary() method behaves differently for full-time and part-time employees.

Code Examples

Question 1

Create a `BankAccount` class that represents a bank account. It should have attributes for account number, account holder name, and balance. Include methods to deposit, withdraw, and display balance. Ensure withdrawals cannot exceed the available balance.

Code

```
class BankAccount:
    def __init__(self, acc_no, name, balance):
        self.acc_no = acc_no
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
        else:
            print("Insufficient balance")

    def display(self):
        print("Balance:", self.balance)


# Sample Input
acc = BankAccount(101, "Ali", 5000)
acc.deposit(2000)
acc.withdraw(1000)
acc.display()
```

```
Balance: 6000
```

Question 2

Design a `Book` class with attributes like title, author, ISBN, and availability status. Create a `Library` class that can add books, search for books by title/author, check out books, and return books. Track which books are currently available.

Code

```python
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.available = True

class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)

    def search(self, name):
        for book in self.books:
            if name.lower() in book.title.lower():
                print(book.title)

    def checkout(self, isbn):
        for book in self.books:
            if book.isbn == isbn and book.available:
                book.available = False
                print("Book Issued")

    def return_book(self, isbn):
        for book in self.books:
            if book.isbn == isbn:
                book.available = True
                print("Book Returned")


# Sample Input
lib = Library()
b1 = Book("Python", "Guido", "111")
lib.add_book(b1)
lib.search("Python")
lib.checkout("111")
lib.return_book("111")
```

```
Python
Book Issued
Book Returned
```

Question 3

Create a `Student` class with attributes for student ID, name, and a list of grades. Implement methods to add grades, calculate average grade, and determine the letter grade (A, B, C, etc.). Add a method to display student information with their performance summary.

Code

```python
class Student:
    def __init__(self, sid, name):
        self.sid = sid
        self.name = name
        self.grades = []

    def add_grade(self, grade):
        self.grades.append(grade)

    def average(self):
        return sum(self.grades) / len(self.grades)

    def letter_grade(self):
        avg = self.average()
        if avg >= 80:
            return "A"
        elif avg >= 70:
            return "B"
        elif avg >= 60:
            return "C"
        else:
            return "F"


# Sample Input
s = Student(1, "Ahmed")
s.add_grade(70)
s.add_grade(80)
print("Average:", s.average())
print("Grade:", s.letter_grade())

# Output
# Average: 75.0
# Grade: B
```

```
Average: 75.0
Grade: B
```

Question 4

Design a `MenuItem` class for food items (name, price, category) and an `Order` class that can add multiple items, calculate total cost, apply discounts, and generate receipts. Include tax calculation functionality.

Code

```python
class MenuItem:
    def __init__(self, name, price):
        self.name = name
        self.price = price

class Order:
    def __init__(self):
        self.items = []

    def add_item(self, item):
        self.items.append(item)

    def total(self):
        return sum(item.price for item in self.items) * 1.1


# Sample Input
item1 = MenuItem("Burger", 300)
item2 = MenuItem("Fries", 200)

order = Order()
order.add_item(item1)
order.add_item(item2)
print("Total Bill:", order.total())

# Output
# Total Bill: 550.0
```

```
Total Bill: 550.0
```

Question 5

Create classes for `Vehicle` (base class) and derived classes `Car`, `Motorcycle`, and `Truck`. Each vehicle should have attributes like license plate, model, rental price per day, and availability. Implement a system to rent vehicles and calculate rental costs.

Code

```python
class Vehicle:
    def __init__(self, plate, price):
        self.plate = plate
        self.price = price
        self.available = True

    def rent(self, days):
        if self.available:
            self.available = False
            return self.price * days


# Sample Input
v = Vehicle("ABC-123", 2000)
print("Rent Cost:", v.rent(3))

# Output
# Rent Cost: 6000
```

```
Rent Cost: 6000
```

Question 6

Design an `Employee` class with attributes for employee ID, name, position, and salary. Create methods to calculate monthly salary, annual salary, and apply raises. Include functionality to track employee details and employment duration.

Code

```python
class Employee:
    def __init__(self, eid, name, salary):
        self.eid = eid
        self.name = name
        self.salary = salary

    def monthly_salary(self):
        print("Monthly Salary:", self.salary)

    def annual_salary(self):
        print("Annual Salary:", self.salary * 12)

    def raise_salary(self, amount):
        self.salary += amount


e = Employee(1, "Sara", 50000)
e.monthly_salary()
e.annual_salary()
e.raise_salary(5000)
e.monthly_salary()
```

```
Monthly Salary: 50000
Annual Salary: 600000
Monthly Salary: 55000
```

Question 7

Create a `Product` class with attributes for product ID, name, price, and stock quantity. Design a `ShoppingCart` class that can add products, remove products, calculate total cost, and handle checkout process while updating inventory.

Code

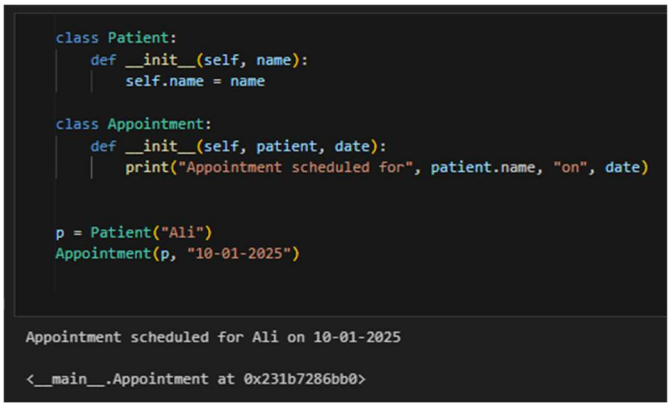
```python
class Product:
    def __init__(self, name, price, stock):
        self.name = name
        self.price = price
        self.stock = stock

class ShoppingCart:
    def __init__(self):
        self.cart = []

    def add(self, product):
        if product.stock > 0:
            self.cart.append(product)
            product.stock -= 1

    def total(self):
        print("Total Cost:", sum(p.price for p in self.cart))


p1 = Product("Laptop", 50000, 2)
cart = ShoppingCart()
cart.add(p1)
cart.add(p1)
cart.total()
```

```
Total Cost: 100000
```

Question 8

Design a `Patient` class to store patient information (ID, name, age, medical history) and an `Appointment` class to manage doctor appointments. Include functionality to schedule, cancel, and view appointments.

Code



```
class Patient:
    def __init__(self, name):
        self.name = name

class Appointment:
    def __init__(self, patient, date):
        print("Appointment scheduled for", patient.name, "on", date)


p = Patient("Ali")
Appointment(p, "10-01-2025")
```

```
Appointment scheduled for Ali on 10-01-2025

<__main__.Appointment at 0x231b7286bb0>
```

Question 9

Create a `User` class for a social media platform with attributes for username, email, friends list, and posts. Implement methods to add friends, create posts, and display user timeline. Include privacy settings for posts.

Code

```
class User:
    def __init__(self, username):
        self.username = username
        self.posts = []

    def post(self, text):
        self.posts.append(text)

    def timeline(self):
        for post in self.posts:
            print(post)


u = User("danish")
u.post("Hello World!")
u.post("Learning Python OOP")
u.timeline()
```

```
Hello World!
Learning Python OOP
```

Question 10

Design a `Room` class with room number, type (single/double/suite), price, and availability. Create a `Booking` class to handle room reservations, check-in/check-out dates, and calculate total stay cost.

Code

```python
class Room:
    def __init__(self, number, price):
        self.number = number
        self.price = price
        self.available = True

class Booking:
    def __init__(self, room, days):
        if room.available:
            room.available = False
            print("Total Cost:", room.price * days)


r = Room(101, 3000)
Booking(r, 2)
```

```
Total Cost: 6000

<__main__.Booking at 0x231b71b4a00>
```

Question 11

Create `Course` classes (with code, name, credits, capacity) and a `Student` class that can register for courses. Implement functionality to check for schedule conflicts and ensure students don't exceed credit limits.

Code

```python
class Course:
    def __init__(self, code, credits):
        self.code = code
        self.credits = credits

class Student:
    def __init__(self, name):
        self.name = name
        self.courses = []

    def register(self, course):
        self.courses.append(course)
        print("Registered:", course.code)


s = Student("Ahmed")
c = Course("CS101", 3)
s.register(c)
```

```
Registered: CS101
```

Question 12

Design a `Product` class for items in inventory (ID, name, quantity, price, supplier) and an `Inventory` class to track stock levels, add new products, update quantities, and generate low-stock alerts.

Code

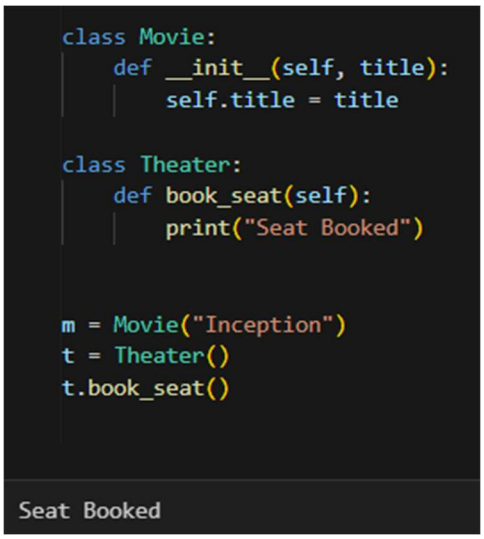
```
class Product:
    def __init__(self, name, qty):
        self.name = name
        self.qty = qty

class Inventory:
    def __init__(self):
        self.products = []

    def add(self, product):
        self.products.append(product)
        print("Added:", product.name)


inv = Inventory()
inv.add(Product("Mouse", 10))
```

```
Added: Mouse
```

Question 13

Create `Movie` classes (title, genre, duration, showtimes) and a `Theater` class with seating arrangement. Implement a booking system that reserves seats and calculates ticket prices with different categories (standard, premium).

Code



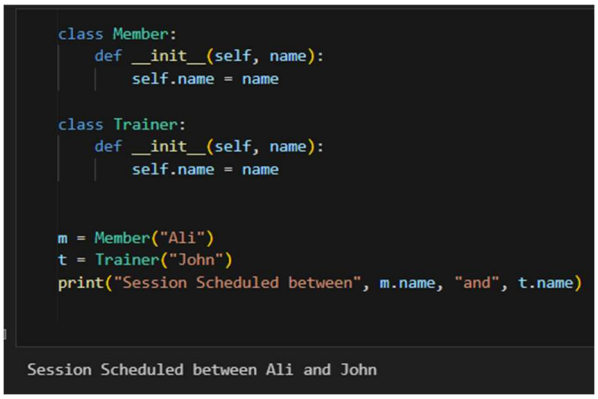
```
class Movie:
    def __init__(self, title):
        self.title = title

class Theater:
    def book_seat(self):
        print("Seat Booked")


m = Movie("Inception")
t = Theater()
t.book_seat()
```

```
Seat Booked
```

Question 14

Design a `Member` class with personal details, membership type (basic/premium), and attendance records. Create a `Trainer` class and a system to schedule training sessions between members and trainers.

Code



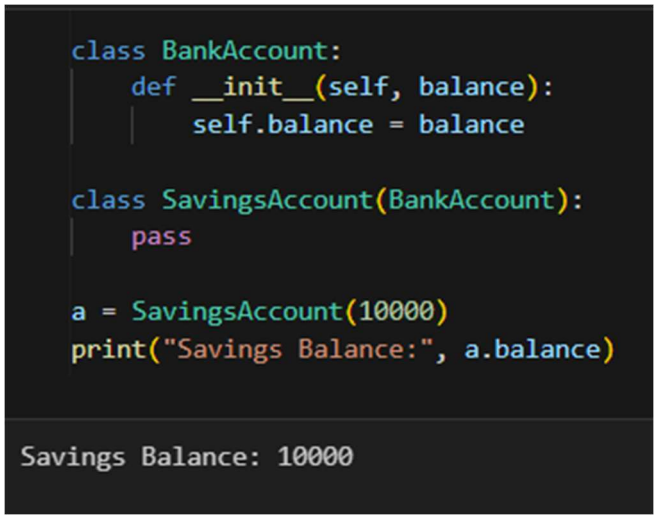
```
class Member:
    def __init__(self, name):
        self.name = name

class Trainer:
    def __init__(self, name):
        self.name = name


m = Member("Ali")
t = Trainer("John")
print("Session Scheduled between", m.name, "and", t.name)
```

```
Session Scheduled between Ali and John
```

Question 15

Extend the bank account system to include `SavingsAccount` and `CheckingAccount` classes with different interest rates and withdrawal rules. Implement fund transfer between accounts.

Code



```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

class SavingsAccount(BankAccount):
    pass

a = SavingsAccount(10000)
print("Savings Balance:", a.balance)
```

```
Savings Balance: 10000
```

Question 16

Create a `Pizza` class with size, crust type, and toppings. Design an ordering system that calculates prices based on selections and allows custom pizza creation with various topping combinations.

Code

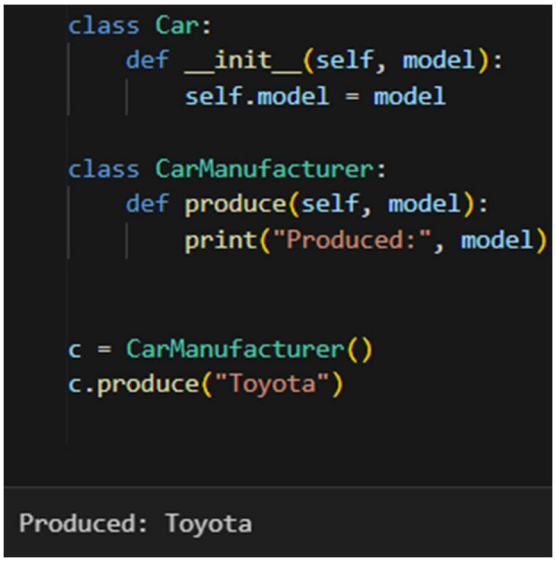
```
class Pizza:
    def __init__(self, size):
        self.price = 500 if size == "small" else 800
        print("Pizza Price:", self.price)


Pizza("large")
```

```
Pizza Price: 800

<__main__.Pizza at 0x231b71b48b0>
```

Question 17

Design classes for `Car` (model, year, color, price) and `CarManufacturer` that can produce different car models with various features. Include quality control checks and production tracking.

Code



```
class Car:
    def __init__(self, model):
        self.model = model

class CarManufacturer:
    def produce(self, model):
        print("Produced:", model)


c = CarManufacturer()
c.produce("Toyota")
```

```
Produced: Toyota
```

Question 18

Create `Teacher` and `Student` classes, and a `Classroom` class that manages assignments, grades, and attendance. Implement functionality to track student performance and generate class reports.

Code

```
class Classroom:
    def __init__(self):
        self.students = []

    def add_student(self, name):
        self.students.append(name)
        print("Added:", name)


c = Classroom()
c.add_student("Ali")
```

```
Added: Ali
```

Question 19

Design `Flight` classes (flight number, destination, departure time, capacity) and a `Passenger` class. Create a booking system that reserves seats and manages passenger manifests.

Code

```
class Flight:
    def __init__(self, number):
        self.number = number
        self.passengers = []

    def book(self, name):
        self.passengers.append(name)
        print("Booked:", name)


f = Flight("PK101")
f.book("Ahmed")
```

```
Booked: Ahmed
```

Question 20

Create classes for `SmartDevice` (base class) and specific devices like `SmartLight`, `Thermostat`, and `SecurityCamera`. Implement a `SmartHome` class that can control all devices and create automation routines.

Code

```
class SmartDevice:
    def on(self):
        print("Device ON")

class SmartHome:
    def __init__(self):
        self.devices = []

    def add_device(self, device):
        self.devices.append(device)
        device.on()


home = SmartHome()
home.add_device(SmartDevice())
```

```
Device ON
```

NumPy Library

What is NumPy

NumPy (Numerical Python) is a powerful Python library used for numerical computing. It provides support for large, multi-dimensional arrays and matrices along with a collection of mathematical functions to operate on them efficiently. NumPy's core is written in C, but it also incorporates C++ for some of its functionality. The use of C and C++ allows NumPy to handle large datasets and perform complex mathematical operations efficiently. The performance benefits are particularly evident in numerical simulations and data analysis tasks.

Main Concepts

ndarray (N-Dimensional Array)
The main data structure in NumPy used to store homogeneous numerical data efficiently.

Array Creation Methods
Creating arrays using functions like array(), zeros(), ones(), arange(), and linspace().

Data Types (dtype)
Each NumPy array has a fixed data type such as integer, float, or complex.

Array Shape and Dimensions
Understanding array size, shape, and number of dimensions using shape, ndim, and size.

Indexing and Slicing
Accessing and modifying elements or sub-arrays efficiently.

Vectorized Operations
Performing element-wise operations without using loops.

Broadcasting
Operating on arrays of different shapes automatically.

Reshaping and Manipulation
Changing array structure using reshape(), flatten(), and transpose().

Universal Functions (ufuncs)
Built-in functions that operate element-wise on arrays (e.g., add, sqrt, exp).

Aggregation and Statistical Functions
Functions like sum(), mean(), min(), max(), and std().

Linear Algebra Operations
Matrix operations such as dot product, inverse, and transpose.

Example Programs

Question 1

A weather station records temperatures (°C) for 7 days:
[21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0]

Task:

- Convert the list into a NumPy array
- Find:
    - Average temperature
    - Maximum and minimum temperature
    - Temperature difference (range)

Code

```
import numpy as np

temps = np.array([21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0])

print("Average:", temps.mean())
print("Max:", temps.max())
print("Min:", temps.min())
print("Range:", temps.max() - temps.min())
```

```
Average: 23.12857142857143
Max: 26.5
Min: 19.8
Range: 6.699999999999999
```

Question 2

A fruit store records daily sales of apples for 4 weeks:

Week 1: [12, 15, 14, 10, 9, 8, 7]

Week 2: [11, 12, 13, 9, 5, 8, 6]

Week 3: [7, 9, 12, 10, 11, 13, 12]

Week 4: [10, 11, 12, 7, 8, 9, 11]

Task:

- Create a **4×7 NumPy array**
- Find weekly totals

- Find the highest sale day (index only)

Find the week with the most sales

Code

```
import numpy as np

sales = np.array([
    [12,15,14,10,9,8,7],
    [11,12,13,9,5,8,6],
    [7,9,12,10,11,13,12],
    [10,11,12,7,8,9,11]
])

weekly_totals = sales.sum(axis=1)

print("Weekly Totals:", weekly_totals)
print("Highest Sale Day Index:", np.argmax(sales))
print("Week with Most Sales:", np.argmax(weekly_totals))
```

```
Weekly Totals: [75 64 74 68]
Highest Sale Day Index: 1
Week with Most Sales: 0
```

Question 3

Students received marks in a test: [56, 78, 45, 90, 88, 76, 59, 62]

Task:

- Convert to NumPy array
- Add 5 bonus marks to every student using broadcasting
- Count how many students now scored > 60
- Find the median and standard deviation

Code

```
import numpy as np

marks = np.array([56, 78, 45, 90, 88, 76, 59, 62])
marks = marks + 5

print("Updated Marks:", marks)
print("Students > 60:", np.sum(marks > 60))
print("Median:", np.median(marks))
print("Standard Deviation:", np.std(marks))
```

```
Updated Marks: [61 83 50 95 93 81 64 67]
Students > 60: 7
Median: 74.0
Standard Deviation: 15.105876340020794
```

Question 4

A smartwatch stores daily step-counts for a user for 30 days.

Task:

- Generate a random NumPy array of shape (30,) with values between 2000–15000
- Reshape into a 5×6 matrix
- Find:
    - Weekly average steps
    - Day with maximum steps
    - Number of days user crossed 10,000 steps

Code

```
import numpy as np

steps = np.random.randint(2000, 15000, 30)
steps = steps.reshape(5, 6)

print("Weekly Average:", steps.mean(axis=1))
print("Max Steps:", steps.max())
print("Days > 10000:", np.sum(steps > 10000))
```

```
Weekly Average: [ 6983.          8604.5        10107.33333333  5512.16666667
 12262.33333333]
Max Steps: 14583
Days > 10000: 9
```

Question 5

Heart rate readings (in BPM) for 10 patients recorded 4 times per day:

- Use array shape = **(10, 4)**

Task:

- Create an integer array using np.random.randint(60,150,(10,4))
- For each patient:
    - Compute daily average
    - Identify abnormal readings > 120

- For all patients:

Compute overall maximum and minimum

Code

```
import numpy as np

heart_rate = np.random.randint(60, 150, (10,4))

print("Daily Average per Patient:", heart_rate.mean(axis=1))
print("Abnormal Readings (>120):", heart_rate[heart_rate > 120])
print("Overall Max:", heart_rate.max())
print("Overall Min:", heart_rate.min())
```

```
Daily Average per Patient: [124.   104.75  97.75 114.   121.   114.5   92.5   95.75 111.5  118.25]
Abnormal Readings (>120): [136 145 123 124 134 147 121 141 132 126 139 132 123 123 139 149]
Overall Max: 149
Overall Min: 69
```

Question 6

Two branches of a store record monthly sales (in thousands):

Branch A: [122, 135, 150, 160, 155, 140]

Branch B: [110, 130, 148, 165, 158, 145]

Task:

- Compute difference between branches
- Calculate:
    - Average monthly sales of each branch
    - Which branch performed better overall?
- Compute correlation between A and B

Code

```
import numpy as np

A = np.array([122,135,150,160,155,140])
B = np.array([110,130,148,165,158,145])

print("Difference:", A - B)
print("Avg A:", A.mean())
print("Avg B:", B.mean())
print("Better Branch:", "A" if A.sum() > B.sum() else "B")
print("Correlation:", np.corrcoef(A, B)[0,1])
```

```
Difference: [12  5  2 -5 -3 -5]
Avg A: 143.66666666666666
Avg B: 142.66666666666666
Better Branch: A
Correlation: 0.9811677353198398
```

Question 7

Given a 4×4 grayscale image:

[[100, 120, 130, 90],

[80 , 110, 140, 100],

[70 , 90 , 120, 130],

[110, 140, 150, 160]]

Task:

- Convert to NumPy array
- Normalize pixel values (0–1)
- Apply a transformation: new_pixel = pixel * 1.2
- Clip all values > 255
- Compute average brightness before & after transformation

Code

```
import numpy as np

image = np.array([
    [100,120,130,90],
    [80,110,140,100],
    [70,90,120,130],
    [110,140,150,160]
])

normalized = image / 255
transformed = np.clip(image * 1.2, 0, 255)

print("Avg Brightness Before:", image.mean())
print("Avg Brightness After:", transformed.mean())
```

```
Avg Brightness Before: 115.0
Avg Brightness After: 138.0
```

Question 8

Dataset contains 100 samples with 4 features each.

Task:

- Generate a 100×4 matrix with values 1–100
- Standardize the dataset using:
- x_scaled = (x - mean) / std
- Perform:
    - Column-wise mean (should be ≈ 0)
    - Column-wise variance (should be ≈ 1)
- Verify results using NumPy functions

Code

```
import numpy as np

data = np.random.randint(1, 101, (100,4))
mean = data.mean(axis=0)
std = data.std(axis=0)

scaled = (data - mean) / std

print("Mean after Scaling:", scaled.mean(axis=0))
print("Variance after Scaling:", scaled.var(axis=0))
```

```
Mean after Scaling: [-9.54791801e-17  8.32667268e-17  1.12132525e-16  7.77156117e-17]
Variance after Scaling: [1. 1. 1. 1.]
```

Question 9

Simulate traffic density (cars per minute) recorded by 6 sensors over 24 hours.

Task:

- Generate array shape (24, 6)
- Find hourly average
- Identify the busiest sensor (highest total density)
- Apply a threshold:

- Replace all values above 50 with 50 (traffic cap)

- Compute energy usage:

energy = Σ(sensor_density × 0.5)

Code

```
import numpy as np

traffic = np.random.randint(10, 80, (24,6))

print("Hourly Average:", traffic.mean(axis=1))
print("Busiest Sensor:", np.argmax(traffic.sum(axis=0)))

traffic = np.clip(traffic, 0, 50)
energy = np.sum(traffic * 0.5)

print("Energy Usage:", energy)
```

```
Hourly Average: [37.83333333 41.5        35.66666667 46.16666667 49.5        42.5
 46.83333333 52.         37.         48.         46.         54.83333333
 51.16666667 31.83333333 31.33333333 46.83333333 42.         33.83333333
 42.33333333 45.5        49.66666667 48.16666667 45.5        56.        ]
Busiest Sensor: 2
Energy Usage: 2749.0
```

Question 10

You are simulating forces in a mechanical system.

Force vectors applied to 5 components:

F = [[5,2,1],

[3,1,4],

[6,3,2],

[4,2,5],

[7,1,3]]

Transformation matrix:

T = [[2,1,0],

[1,3,1],

[0,2,2]]

Task:

- Compute transformed forces: F_new = F × T
- Compute magnitude of each force vector
- Identify the component experiencing highest transformed force
- Perform eigenvalue decomposition of T

Code

```python
import numpy as np

F = np.array([
    [5,2,1],
    [3,1,4],
    [6,3,2],
    [4,2,5],
    [7,1,3]
])

T = np.array([
    [2,1,0],
    [1,3,1],
    [0,2,2]
])

F_new = F @ T
magnitudes = np.linalg.norm(F_new, axis=1)

print("Transformed Forces:\n", F_new)
print("Magnitudes:", magnitudes)
print("Highest Force Component:", np.argmax(magnitudes))
print("Eigenvalues of T:", np.linalg.eig(T)[0])
```

```
Transformed Forces:
 [[12 13  4]
 [ 7 14  9]
 [15 19  7]
 [10 20 12]
 [15 16  7]]
Magnitudes: [18.13835715 18.05547009 25.19920634 25.37715508 23.02172887]
Highest Force Component: 3
Eigenvalues of T: [4.30277564 2.         0.69722436]
```

Seaborn Library

What is Seaborn

Seaborn is a popular Python data visualization library built on top of Matplotlib. It provides a high-level interface for creating attractive and informative statistical graphics with minimal code, and integrates seamlessly with Pandas DataFrames.

Key Features and Uses

- High-level API: Seaborn simplifies the process of creating complex plots that would require more boilerplate code in Matplotlib.
- Statistical Plotting: The library focuses on visualizing relationships, distributions, and trends in data and includes functions for specific statistical plot types like regression plots and heatmaps.
- Aesthetics and Themes: Seaborn offers beautiful default styles, built-in themes (e.g., darkgrid, whitegrid), and customized color palettes, making it easy to create visually appealing plots.
- Integration with Pandas: It works well with pandas data structures, often accepting the DataFrame directly and using column names for axis labels.

Common Plot Types

Seaborn supports a wide variety of plot types, including:

- Relational Plots: scatterplot(), lineplot() for visualizing relationships between numerical variables.
- Categorical Plots: barplot(), countplot(), boxplot(), violinplot(), swarmplot() for visualizing data across different categories.
- Distribution Plots: histplot(), kdeplot(), displot() for examining univariate and bivariate distributions of data.
- Matrix Plots: heatmap(), clustermap() for visualizing data in a color-encoded matrix, often used for correlation matrices.
- Regression Plots: regplot(), lmplot() for plotting a linear regression model fit along with data points.
- Multi-plot Grids: Functions like FacetGrid, pairplot(), and jointplot() for creating multiple plots in a grid layout to explore relationships among several variables.

Example Programs

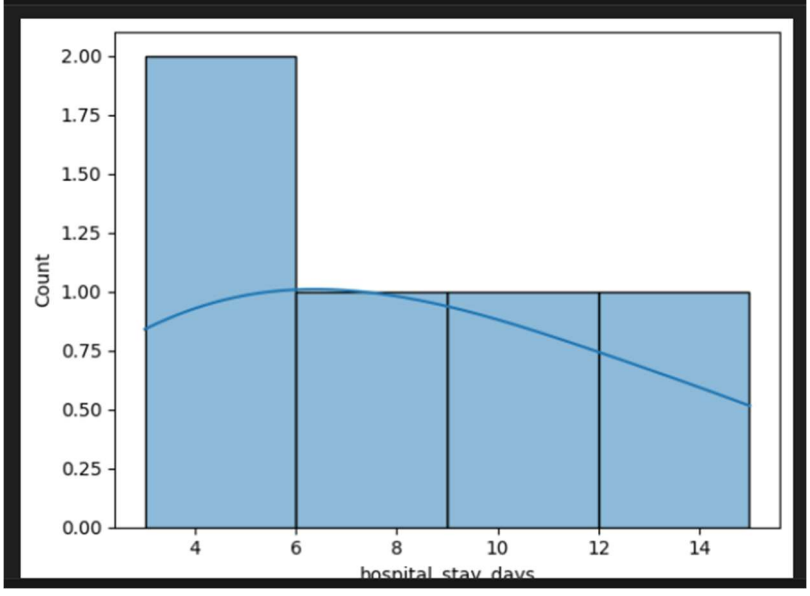
Q1. Patient Stay Analysis (Healthcare)

You are given a dataset containing age, disease_type, and hospital_stay_days.
Write a Python program using **Seaborn** to:

- Plot the distribution of hospital stay days
- Compare hospital stay across different disease types
- Identify which disease has the highest median stay

Code

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.DataFrame({
    'age': [25, 40, 60, 30, 50],
    'disease_type': ['Flu', 'Covid', 'Flu', 'Cancer', 'Covid'],
    'hospital_stay_days': [3, 10, 4, 15, 8]
})

sns.histplot(df['hospital_stay_days'], kde=True)
plt.show()

sns.boxplot(x='disease_type', y='hospital_stay_days', data=df)
plt.show()

print(df.groupby('disease_type')['hospital_stay_days'].median().idxmax())
```



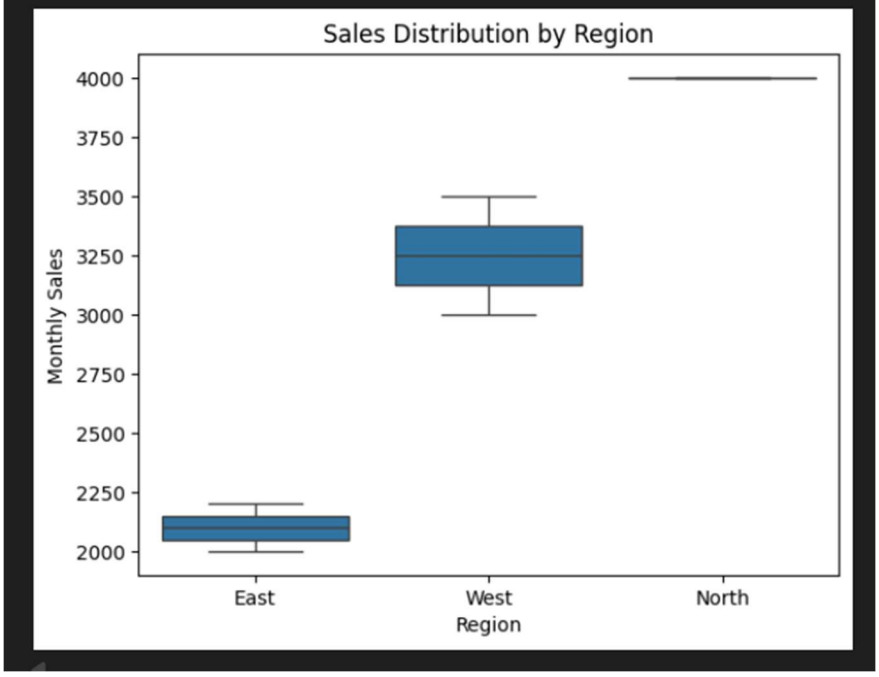
Q2. Sales Distribution Comparison (Business)

A retail dataset contains region and monthly_sales.
Write code to:

- Visualize sales distribution per region using Seaborn
- Detect regions with high sales variability
- Label the axes and add a title

Code

```
df = pd.DataFrame({
    'region': ['East', 'West', 'East', 'North', 'West'],
    'monthly_sales': [2000, 3500, 2200, 4000, 3000]
})

sns.boxplot(x='region', y='monthly_sales', data=df)
plt.xlabel('Region')
plt.ylabel('Monthly Sales')
plt.title('Sales Distribution by Region')
plt.show()

print(df.groupby('region')['monthly_sales'].std())
```



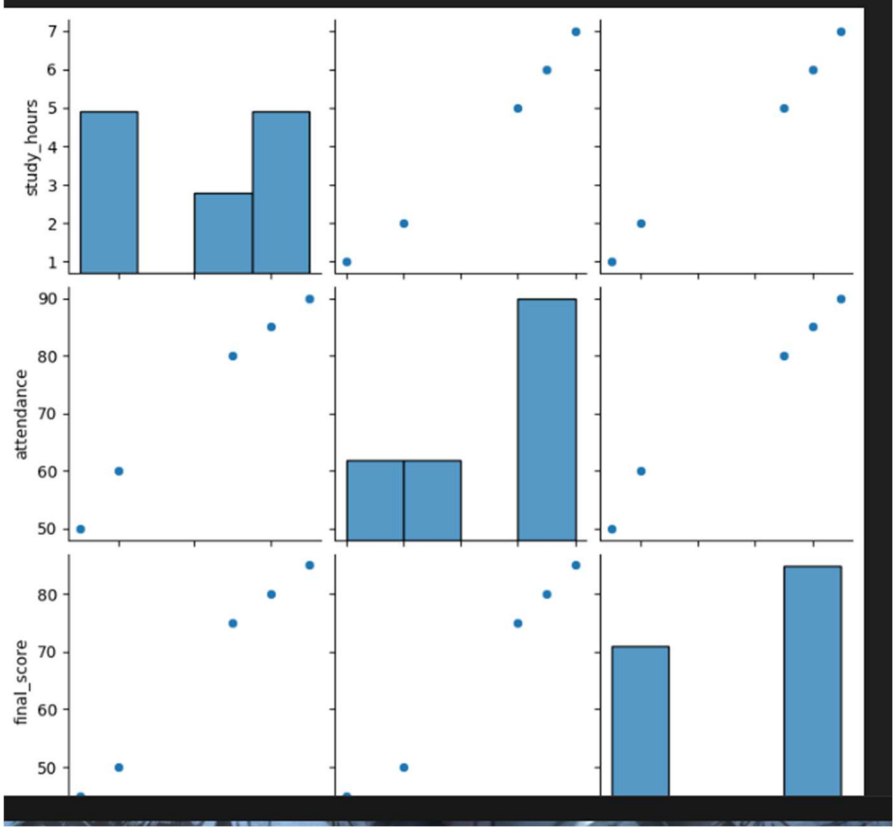
Q3. Student Performance Correlation (Education)

A dataset contains study_hours, attendance, and final_score .
Using Seaborn:

- Create a pair plot
- Plot a correlation heatmap
- Interpret which feature most affects the final score

Code

```
df = pd.DataFrame({
    'study_hours': [2, 5, 7, 1, 6],
    'attendance': [60, 80, 90, 50, 85],
    'final_score': [50, 75, 85, 45, 80]
})

sns.pairplot(df)
plt.show()

sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()

print(df.corr()['final_score'])
```



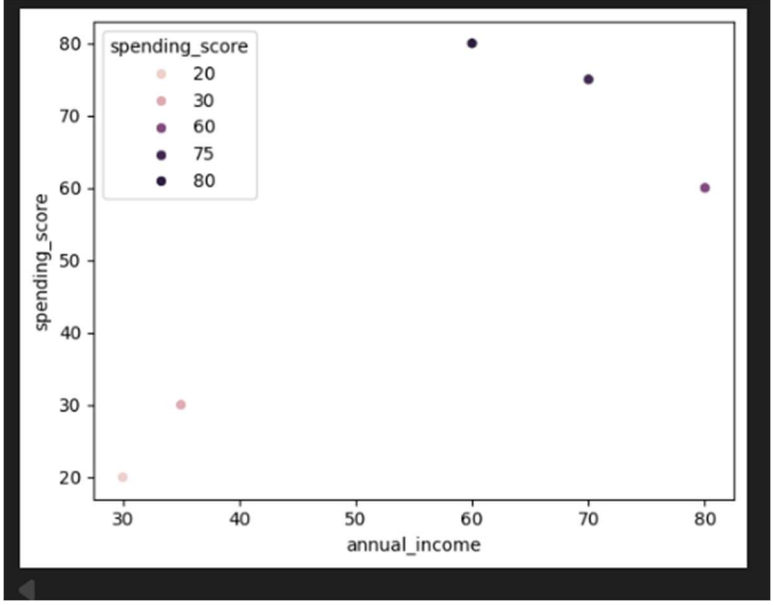
Q4. Customer Segmentation Visualization

Customer data includes age, annual_income, and spending_score.
Write a program to:

- Visualize relationships using Seaborn scatter plots
- Color-code points based on spending score
- Identify potential customer segments visually

Code

```
df = pd.DataFrame({
    'age': [22, 35, 45, 25, 40],
    'annual_income': [30, 60, 80, 35, 70],
    'spending_score': [20, 80, 60, 30, 75]
})

sns.scatterplot(
    x='annual_income',
    y='spending_score',
    hue='spending_score',
    data=df
)
plt.show()
```



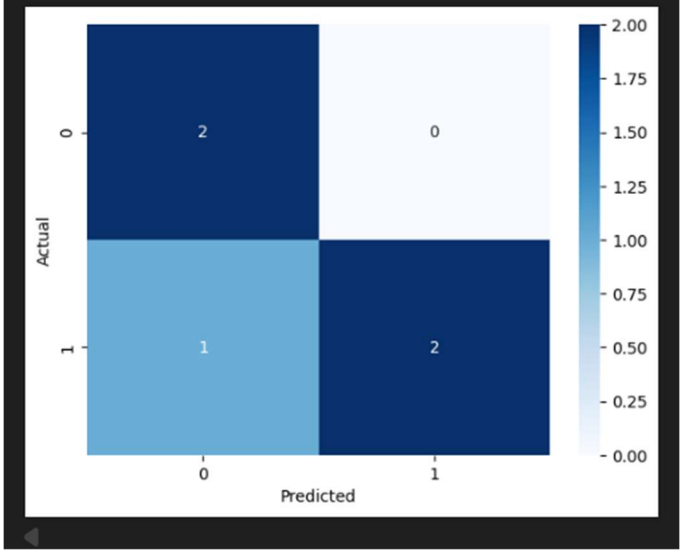
Q5. Model Result Visualization

You trained a classifier and obtained y_true and y_pred.
Write code using Seaborn to:

- Plot a confusion matrix heatmap
- Add annotations and color bar
- Clearly label predicted vs actual classes

Code

```python
import numpy as np
from sklearn.metrics import confusion_matrix

y_true = [0, 1, 0, 1, 1]
y_pred = [0, 1, 0, 0, 1]

cm = confusion_matrix(y_true, y_pred)

sns.heatmap(cm, annot=True, cmap='Blues', fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



Scikit Learn Library

What is Scikit Learn

Scikit-learn **(sklearn)** is a widely-used, free, and open-source machine learning library for the Python programming language. It provides efficient tools for classical machine learning and statistical modeling, including classification, regression, clustering, and dimensionality reduction.

Key Features and Capabilities

- **Comprehensive Algorithms:** Scikit-learn offers a vast array of algorithms for both supervised and unsupervised learning:
    - **Classification:** Support Vector Machines (SVMs), Random Forests, Logistic Regression, Gradient Boosting, Naive Bayes, k-Nearest Neighbors (KNN).
    - **Regression:** Linear Regression, Ridge Regression, Support Vector Regression, Decision Tree Regression.
    - **Clustering:** K-Means, DBSCAN, Hierarchical clustering.
    - **Dimensionality Reduction:** Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA), Non-Negative Matrix Factorization (NMF).
- **Data Preparation & Model Evaluation:** The library includes robust utilities for data preprocessing (feature scaling, normalization, handling missing values, encoding categorical data), model selection (cross-validation, grid search for hyperparameter tuning), and evaluation metrics (accuracy, precision, recall, F1-score, ROC/AUC).
- **Consistent API:** It is known for its simple and consistent API, using the standard fit(X, y) method for training models and predict(T) for making predictions, which makes it easy to learn and use across different algorithms.
- **Interoperability:** Scikit-learn is built on top of other core Python scientific libraries, such as NumPy, SciPy, and Matplotlib, allowing it to integrate seamlessly into a standard data science workflow for data manipulation and visualization

Example Programs

Q1. Telecom Customer Churn Prediction

Given customer data with features such as tenure, monthly_charges, and contract_type, and target churn.
Write a Scikit-learn program to:

- Encode categorical variables

- Train a Logistic Regression model
- Evaluate using accuracy and confusion matrix

Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# Sample dataset
df = pd.DataFrame({
    'tenure': [1, 5, 10, 20, 3, 8],
    'monthly_charges': [30, 70, 60, 90, 40, 80],
    'contract_type': ['Month-to-month', 'One year', 'Two year', 'Month-to-month', 'One year', 'Two year'],
    'churn': [1, 0, 0, 1, 1, 0]
})

X = df[['tenure', 'monthly_charges', 'contract_type']]
y = df['churn']

# Encode categorical variable
X = pd.get_dummies(X, drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

✓ 0.0s

```
Accuracy: 0.0
Confusion Matrix:
 [[0 0]
 [2 0]]
```

Q2. House Price Prediction

A dataset contains area, bedrooms, location_score, and price.
Implement a regression model to:

Split data into train/test sets, Train Linear Regression and Predict prices and calculate Mean Squared Error

Code

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Sample dataset
df = pd.DataFrame({
    'area': [800, 1000, 1200, 1500, 1800],
    'bedrooms': [1, 2, 2, 3, 4],
    'location_score': [6, 7, 8, 9, 9],
    'price': [200000, 250000, 300000, 400000, 500000]
})

X = df[['area', 'bedrooms', 'location_score']]
y = df['price']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
print("Mean Squared Error:", mean_squared_error(y_test, predictions))
```

✓ 0.0s

```
Mean Squared Error: 156250000.00005385
```

Q3. Disease Classification System

Patient data includes glucose, BMI, age, and outcome.
Write code to:

- Normalize the data
- Train a Decision Tree classifier
- Evaluate precision, recall, and F1-score

Code

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import precision_score, recall_score, f1_score

# Sample dataset
df = pd.DataFrame({
    'glucose': [120, 140, 90, 160, 110],
    'BMI': [25, 30, 22, 35, 28],
    'age': [25, 45, 30, 50, 40],
    'outcome': [0, 1, 0, 1, 1]
})

X = df[['glucose', 'BMI', 'age']]
y = df['outcome']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2)

model = DecisionTreeClassifier()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1-score:", f1_score(y_test, y_pred))
```

✓ 0.0s

```
Precision: 0.0
Recall: 0.0
F1-score: 0.0
```

Q4. Email Spam Detection

You are given email text and spam labels.
Using Scikit-learn:

- Convert text to numerical features (TF-IDF)
- Train a Naive Bayes classifier
- Test the model on new email text

Code

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

# Sample dataset
df = pd.DataFrame({
    'email_text': [
        'Win a free prize now',
        'Meeting scheduled tomorrow',
        'Congratulations you won lottery',
        'Please find the attached report'
    ],
    'spam': [1, 0, 1, 0]
})

X = df['email_text']
y = df['spam']

vectorizer = TfidfVectorizer()
X_tfidf = vectorizer.fit_transform(X)

model = MultinomialNB()
model.fit(X_tfidf, y)

new_email = ["Free lottery win now"]
new_vec = vectorizer.transform(new_email)
print("Spam Prediction:", model.predict(new_vec))
```

✓ 0.0s

```
Spam Prediction: [1]
```

Q5. Student Risk Prediction

Student records include attendance, mid_marks, and final_marks.
Write a program to:

- Build a Random Forest classifier
- Predict whether a student is “At Risk”
- Display feature importance

```python
import pandas as pd
from sklearn.ensemble import RandomForestClassifier

# Sample dataset
df = pd.DataFrame({
    'attendance': [60, 85, 70, 90, 50],
    'mid_marks': [40, 75, 55, 80, 35],
    'final_marks': [45, 78, 60, 85, 40],
    'at_risk': [1, 0, 1, 0, 1]
})

X = df[['attendance', 'mid_marks', 'final_marks']]
y = df['at_risk']

model = RandomForestClassifier()
model.fit(X, y)

print("Feature Importance:")
for feature, importance in zip(X.columns, model.feature_importances_):
    print(feature, ":", importance)
```

✓ 0.3s

```
Feature Importance:
attendance : 0.3541666666666667
mid_marks : 0.3125
final_marks : 0.3333333333333333
```

TensorFlow Library

What is TensorFlow?

TensorFlow is an **open-source library developed by Google** for machine learning (ML) and deep learning (DL). It is widely used to build and train neural networks for tasks like image recognition, natural language processing, and predictive modeling.

Key Features of TensorFlow:

- **Multi-platform:** Runs on CPU, GPU, mobile devices, and even browsers.
- **Flexible:** Supports high-level APIs like Keras for easy model building.
- **Scalable:** Can handle large datasets and distributed training.
- **Visualization:** Comes with TensorBoard for monitoring and visualizing model performance.

Where is TensorFlow used?

TensorFlow is used in:

- Image recognition
- Face detection
- Voice assistants
- Text classification
- Chatbots
- Medical analysis
- Recommendation systems (like Netflix)
- Self-driving cars

Code Examples

Basic Training

```python
import tensorflow as tf
from tensorflow import keras
import numpy as np

# Simple model with one hidden layer
model = keras.Sequential([
    keras.layers.Dense(10, activation='relu', input_shape=(1,)),  # Specify input shape
    keras.layers.Dense(1)
])

# Compile the model
model.compile(optimizer='adam', loss='mse')

# Convert to NumPy arrays and reshape
x = np.array([1, 2, 3, 4], dtype=np.float32).reshape(-1, 1)
y = np.array([2, 4, 6, 8], dtype=np.float32).reshape(-1, 1)

# Train the model
model.fit(x, y, epochs=50, verbose=0)  # verbose=0 to hide training output

# Predict - need to reshape the input for prediction too
print(model.predict(np.array([5]).reshape(-1, 1)))
```

Linear Regression

```python
# Import necessary libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Prepare Data
# Features (X) and labels (Y)
X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float)  # Linear relationship: y = 2*x

# Step 2: Build the Model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1, input_shape=[1])  # 1 input, 1 output
])

# Step 3: Compile the Model
model.compile(
    optimizer='sgd',          # Stochastic Gradient Descent optimizer
    loss='mean_squared_error' # Loss function
)

# Step 4: Train the Model
history = model.fit(X, Y, epochs=1000, verbose=0)  # Train silently

# Step 5: Make Predictions
new_X = np.array([6, 7, 8], dtype=float)
predictions = model.predict(new_X)
print("Predictions for [6, 7, 8]:", predictions.flatten())

# Step 6: Visualize the Results
plt.scatter(X, Y, color='blue', label='Original Data')  # Original points
plt.plot(X, model.predict(X), color='red', label='Fitted Line')  # Regression line
plt.scatter(new_X, predictions, color='green', label='Predictions')  # Predictions
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression using TensorFlow")
plt.legend()
plt.show()
```